IoT Device Forensic Plan: Investigation Memo

(SYNTHETIC Case Evaluation & Reproducible Test)

Student Name:	DEEPTANSH NAGAR	**Student ID:**	500121033
Project Code:	DF-PBL-026	**Performance Mode:**	Investigation Memo
Evaluation Date:	October 27, 2026	**Status:**	Final Complete Submission

Executive Overview

This investigation memo documents the forensic acquisition planning, chain-of-custody integrity verification, and analytical hypothesis testing for an incident involving an unauthorized configuration change and video uploads on a smart security camera. All data, devices, and logs analyzed in this report are **SYNTHETIC**.

The investigation follows defensible digital forensics practices aligned with NIST SP 800-86 and ISO/IEC 27037: strictly segregating observation from inference, evaluating competing hypotheses via an evidence matrix, enforcing cryptographic integrity gating (SHA-256), and verifying test reproducibility through an automated Python evidence management utility.

Rubric Criterion	Weight	Where Addressed in This Memo
Problem Framing & Reasoning	20%	Section 1: Scenario framing, precise forensic question, 4 competing hypotheses (H1–H4), expected separation patterns.
Technical Execution	25%	Sections 2, 3, and 5: Architecture map, acquisition plan, on/off decision tree, `evidence_tool.py` execution.
Evidence Integrity	15%	Sections 3.1, 3.4, and 5: RFC 3227 order of volatility, SHA-256 hashing, chain-of-custody, tamper-gate test.
Analysis & Limitations	20%	Sections 4 and 6: Evidence matrix (M1–M5), pending items (P1–P6), technical limitations (H1 untested).
Deliverable Quality	10%	Structured investigation memo format, detailed component tables, risk register, clean typography.
Defense & Originality	10%	Sections 7 and 9: Defensible conclusions without overclaiming, verified sources, authentic student AI declaration.

1. Problem Statement & Hypotheses

1.1 Synthetic Incident Scenario

An office administrator reported that a deployed smart camera uploaded three video clips between 02:14 and 02:31 while the facility was closed and unoccupied. The following morning, the owner discovered that motion detection notifications had been switched to **OFF**. The owner stated they did not disable the notifications and did not authorize video clip exports.

```
================================== SYNTHETIC INCIDENT TIMELINE ==================================

Day 1, 18:40:00 (UTC 13:10)   Day 1, 23:55:02 (UTC 18:25)   Day 2, 02:10:07 (UTC 20:40)   Day 2, 02:14 - 02
[ Office Closes / Armed ] --> [ New Cloud App Session ] -> [ Alert Changed to OFF ] --> [ 3 Clips Uploaded
  camerad: mode=night           session=S-0002 known=false   Cloud API cmd_received        17-minute cluste
=================================================================================================
```

Specification Category	System Parameter (SYNTHETIC)	Forensic Relevance
Target Device	"LumiCam C2" Indoor Security Camera	Subject edge device containing volatile RAM and non-volatile flash.
Hardware Platform	Embedded 32-bit ARM SoC, 128 MB RAM, 16 MB SPI Flash	Determines volatile memory volatility and acquisition interfaces.
Interfaces	Wi-Fi 802.11 b/g/n (2.4 GHz), 3.3V UART header, MicroSD slot	Physical and logical vectors for extraction (D1, D6).
Firmware & OS	Embedded Linux v2.3.1 (Kernel 4.9.x, BusyBox)	Defines running process lists and standard daemon behaviors.
Cloud & App Endpoints	`api.lumicam.example` , `storage.lumicam.example`	RFC 2606 reserved documentation domains handling API commands and video storage.
Local Network	`192.0.2.0/24` , Camera IP: `192.0.2.45`	RFC 5737 documentation prefix isolating local Wi-Fi state.

1.2 Forensic Question & Boundaries

> ***Forensic Question:*** *Based on the evidence that can be lawfully acquired from the synthetic LumiCam C2 camera, its local network environment, its companion application, and associated cloud service records, what best explains the modification of the motion alert setting to OFF at 02:10 and the three video clip uploads between 02:14 and 02:31 on Day 2, and with what level of certainty?*

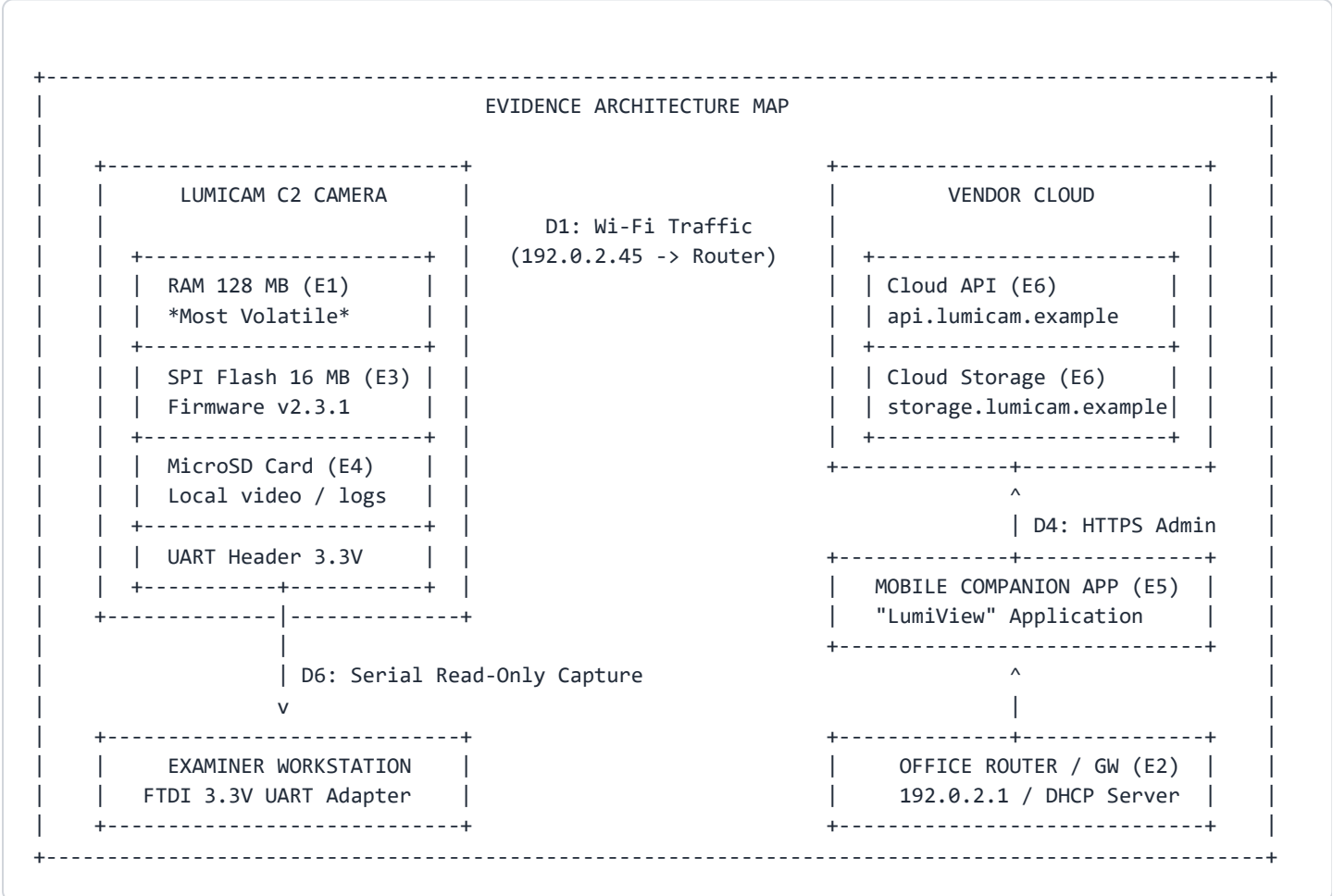
In Scope: Technical examination of volatile memory on the live camera, non-volatile onboard flash storage, removable storage (MicroSD), router transaction logs and DHCP leases, mobile companion application artifacts, and vendor cloud API access logs.

Out of Scope: Identification of real-world human actors, legal attribution of motive or intent, and investigation of any real third-party devices, cloud accounts, or proprietary networks.

1.3 Competing Alternative Hypotheses

- **H1 (Device Compromise):** The camera's local software, runtime memory, or firmware was altered by an unauthorized party via local serial interface, network vulnerability, or unauthorized executable, bypassing cloud authorization.
- **H2 (Cloud Account Misuse):** An unauthorized party compromised the owner's cloud credentials or hijacked an authentication token (reflected by `session=S-0002`) to dispatch valid management commands through the cloud API.
- **H3 (Benign Malfunction):** A software fault, cloud synchronization retry race condition, or firmware defect altered the setting and initiated clip uploads without malicious intervention.
- **H4 (Authorized User Action):** An authorized user with legitimate access (e.g. secondary family member or keyholder) altered the setting and forgot or failed to inform the primary owner.

2. Evidence Inventory & Volatility Hierarchy



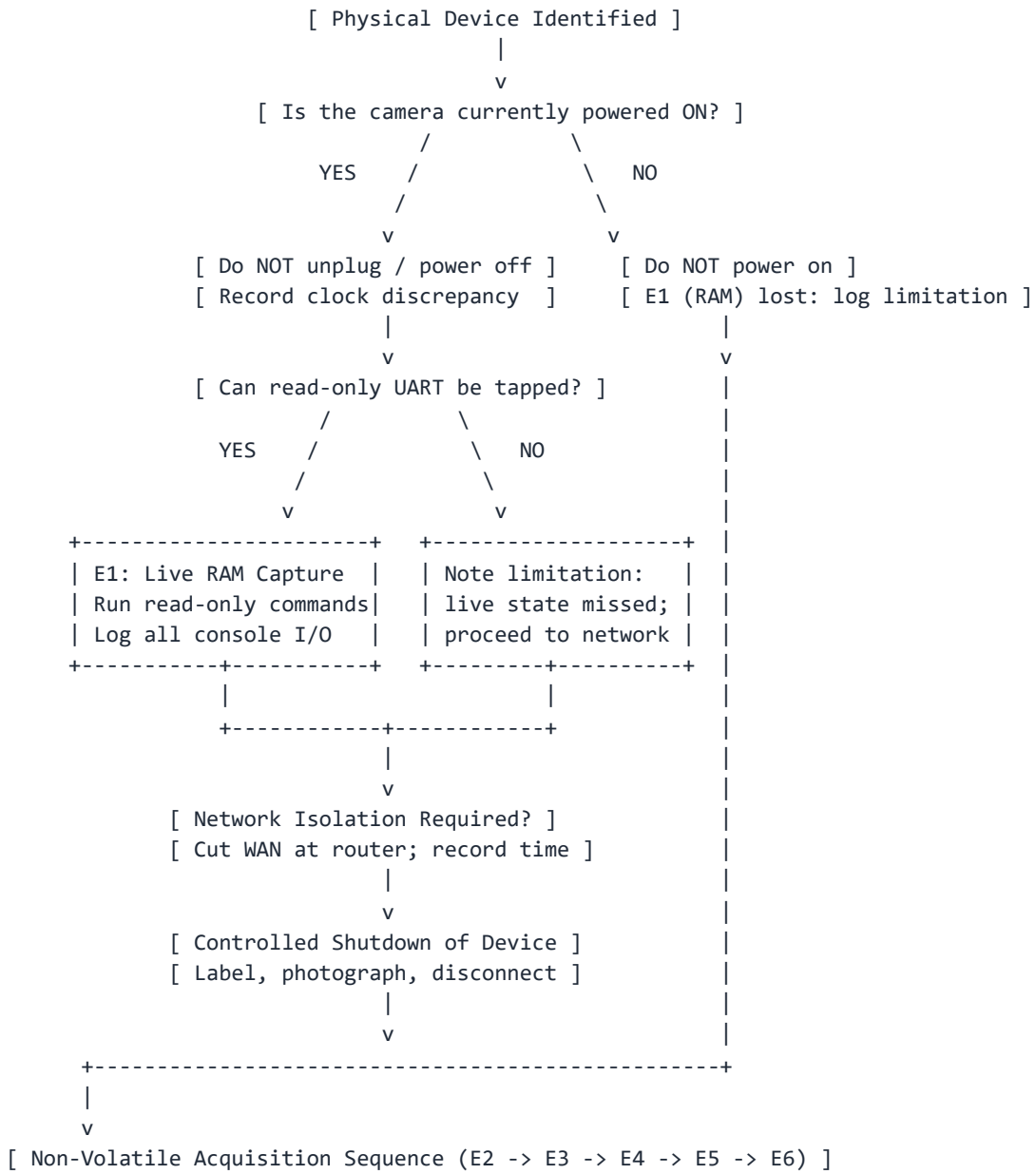
2.1 Order of Volatility (RFC 3227)

Evidence acquisition strictly follows the volatility order defined in **RFC 3227**:

Order	ID	Evidence Location	Volatility	Risk if Collection Delayed
1	**E1**	Camera RAM & Active Sockets	HIGH	Instantly lost upon power interruption; overwritten during ongoing execution.
2	**E2**	Router State & DHCP/NAT Tables	MEDIUM-HIGH	Overwritten by circular logging or state expiration under high traffic.
3	**E5**	Companion App Local Cache	MEDIUM	Subject to automatic cloud synchronization overwrites upon app launch.
4	**E6**	Cloud API & Storage Telemetry	MEDIUM	Subject to vendor retention windows (often 7–30 days) and account modification.
5	**E4**	MicroSD Video & Offline Cache	LOW	Non-volatile, but can be overwritten by circular ring-buffer recording if left powered on.
6	**E3**	Camera SPI NOR Flash (Firmware)	LOWEST	Highly persistent. Low risk if extracted with appropriate hardware tools.

3. Acquisition Plan & Risk Management

3.1 On/Off Decision Tree



3.2 Acquisition Plan & Tooling Matrix

ID	Source	Method & Tooling (Planned)	Expected Artifact	Validation Protocol
E1	Camera RAM	Serial terminal connection (3.3V UART). Run read-only commands (`ps` , `netstat` , `dmesg`). *Tool: picocom / minicom, FTDI adapter (Planned tool, not run).*	`console_ram_capture.txt`	Calculate SHA-256 immediately upon closing terminal capture.
E2	Network State	Administrative export of DHCP leases, NAT states, and DNS queries. *Tool: Router Admin Export, Wireshark (Planned tool, not run).*	`router_dhcp_nat.log`	Calculate SHA-256 immediately upon export from router.
E3	SPI NOR Flash	Controlled shutdown; clip 8-pin SOIC clip to SPI flash; read twice. *Tool: flashrom v1.2+, CH341A programmer (Planned tool, not run).*	`spi_flash_image.bin`	Calculate SHA-256 of Read 1; must match Read 2 bit-for-bit.
E4	MicroSD Card	Physical card removal; mount via hardware write-blocker; bit-stream image. *Tool: Tableau write-blocker, dc3dd / FTK Imager (Planned tool, not run).*	`microsd_image.raw`	SHA-256 hash of card must equal SHA-256 of output image.
E5	Mobile App	Consensual logical backup of smartphone running LumiView app. *Tool: Android Backup Extractor (Planned tool, not run).*	`lumiview_app_data.tar`	Calculate SHA-256 of extraction archive container.
E6	Cloud Records	Formal compliance export request for OAuth tokens, API calls, and video metadata. *Tool: Vendor Compliance Portal (Planned procedure, not run).*	`cloud_api_audit.json`	Calculate SHA-256 immediately upon secure electronic delivery.

3.3 Risk Register

ID	Risk Description	Score (L×I)	Mitigation Strategy	Residual Risk
R1	Volatile RAM lost at power cut	6 (2×3)	Keep device connected to UPS; label "DO NOT UNPLUG"; capture E1 over UART prior to shutdown.	Medium (RAM alters as device continues running).
R2	Remote wipe / cloud sync overwrite	6 (2×3)	Capture live network state (E2), then immediately block WAN routing at router level; request cloud preservation early.	Medium (Network isolation disrupts active connections).
R4	Hardware encryption or locked bootloader	6 (2×3)	Check UART for root console prompt; if locked, pivot to non-volatile storage analysis (E2, E4, E5, E6).	High (Hardware-level evidence may remain inaccessible).
R5	Cloud legal & authorization limits	9 (3×3)	Obtain written consent from account holder prior to analysis; request formal preservation letter to cloud vendor immediately.	Medium (Vendor may reject request or prune logs after retention window).
R7	Evidence contamination / alteration	6 (2×3)	Employ physical write-blockers; immediately baseline raw evidence with SHA-256; perform all parsing on working copies.	Low.
R8	Clock drift and timezone confusion	9 (3×3)	Compare device hardware clock with examiner NTP reference; convert all logged times to UTC while preserving original strings.	Medium (Hardware RTC may have drifted non-linearly).

4. Analysis & Evidence Matrix

ID	Factual Observation	Technical Interpretation	Forensic Inference	Supports / Contradicts	Confidence
M1	Cloud log records login at Day 1 23:55:02 UTC+05:30 with `session=S-0002` `known=false` .	A new web/mobile session was authenticated against the cloud account.	An entity possessing valid credentials logged into the cloud; does not identify who controlled the session.	**Weakly supports H2, H4**; Neutral for H1, H3	LOW
M2	Cloud API log records `CMD_RECEIVED` for `cmd=set_motion_alert value=off session=S-0002` at Day 2 02:10:07, matching camera `CONFIG_CHANGE` .	The setting was updated via an authenticated cloud command received by the device.	The setting change can be explained entirely through cloud API messaging without requiring local device exploitation.	**Consistent with H2, H4, H3** (if automated); Does not require H1	LOW-MED
M3	Camera uploaded 3 video clips to `storage.lumicam.example` between 02:14:11 and 02:31:15.	The camera transmitted video payloads to the vendor's designated cloud storage endpoint.	Video uploads commenced 4 minutes after the configuration change. Demonstrates a temporal sequence, not causation.	**Compatible with all four** (H1, H2, H3, H4)	LOW
M4	The office owner stated the office was unoccupied and they did not disable alerts or view clips.	The registered account holder disclaims initiating the configuration change or video export.	Weakly suggests the owner did not knowingly make the change, but cannot rule out forgotten actions or shared credentials.	**Weakly contradicts H4**; Consistent with H1, H2, H3	LOW
M5	The camera was still powered on and operational at 09:05 when	Volatile memory (E1) was not	Live volatile state could	**Neutral** across all	MEDIUM

ID	Factual Observation	Technical Interpretation	Forensic Inference	Supports / Contradicts	Confidence
	reported.	wiped by power cycling prior to reporting.	theoretically be acquired if examined prior to power loss.	hypotheses	

4.1 Pending Artifacts (P1–P6)

- **P1 (Cloud Session IP & Fingerprint):** Would determine whether session S-0002 originated from an unknown foreign IP (H2) or a known user handset (H4).
- **P2 (Cloud Command Token Audit):** Would determine whether the 02:10 command was dispatched by user session S-0002 or a backend maintenance task (H3).
- **P3 (Camera Console & Process List):** Would determine whether unauthorized binaries were running on the camera (H1).
- **P4 (SPI Flash Gold Hash Comparison):** Would establish whether device firmware was modified (H1).
- **P5 (Router Network PCAP):** Would identify whether the camera communicated with non-vendor exfiltration addresses (H1).
- **P6 (Mobile App Cache):** Would confirm whether the owner's handset holds session records for S-0002 (H4 vs H2).

5. Reproducible Test (Technical Execution)

The reproducible test tool (`evidence_tool.py`) was executed in PowerShell on Windows using **Python 3.11.9**. The tool enforces cryptographic hash gating, refusing to parse any evidence file that deviates from its baseline.

5.1 Real Execution Outputs (Captured from Local Machine)

1. Initializing Baseline (`init`):

```
python evidence_tool.py init
HASHED  camera_console_synthetic_A.log  ad1d0c5f2b7c80aff2ff6739173e5f14369561eb020b51d2476fdc495f
HASHED  camera_console_synthetic_B.log  36f1c40006de149f172b7a9ae8c70911aadc02f3713e3c2e7aa6183dd5
Baseline written to evidence_log.json
```

2. Baseline Verification (`verify`):

```
python evidence_tool.py verify
OK        camera_console_synthetic_A.log
OK        camera_console_synthetic_B.log
OVERALL: PASS - all hashes match
```

3. Parsing Synthetic Log A (`parse`):

```
python evidence_tool.py parse --file evidence/camera_console_synthetic_A.log
File: camera_console_synthetic_A.log
SHA-256 now : ad1d0c5f2b7c80aff2ff6739173e5f14369561eb020b51d2476fdc495fe8ec4a
Integrity   : OK

Parsed 11 event(s); 1 malformed line(s) at line(s) [14]

Timeline (sorted by UTC)
UTC time             Local time as logged       Component    Event            Fields
2026-01-10 13:10:00  2026-01-10T18:40:00+05:30  camerad      ARMED            mode=night
2026-01-10 18:25:02  2026-01-10T23:55:02+05:30  cloud-agent  SESSION_SEEN     session=S-0002 known
2026-01-10 20:40:07  2026-01-11T02:10:07+05:30  cloud-agent  CMD_RECEIVED     cmd=set_motion_alert
2026-01-10 20:40:07  2026-01-11T02:10:07+05:30  config       CONFIG_CHANGE    key=motion_alert old
2026-01-10 20:44:11  2026-01-11T02:14:11+05:30  uploader     UPLOAD_START     clip=clip_001 dest=s
2026-01-10 20:44:40  2026-01-11T02:14:40+05:30  uploader     UPLOAD_DONE      clip=clip_001 dest=s
2026-01-10 20:52:05  2026-01-11T02:22:05+05:30  uploader     UPLOAD_START     clip=clip_002 dest=s
2026-01-10 20:52:33  2026-01-11T02:22:33+05:30  uploader     UPLOAD_DONE      clip=clip_002 dest=s
2026-01-10 21:00:48  2026-01-11T02:30:48+05:30  uploader     UPLOAD_START     clip=clip_003 dest=s
2026-01-10 21:01:15  2026-01-11T02:31:15+05:30  uploader     UPLOAD_DONE      clip=clip_003 dest=s
2026-01-11 03:35:00  2026-01-11T09:05:00+05:30  camerad      STATUS           state=running

Gap, first CONFIG_CHANGE to first UPLOAD_START: 4 min 4 s (timing only, not proof of cause)

Observation flags (consistent-with only, NOT proof)
  line 5   UNRECOGNISED_SESSION  session=S-0002 | A session is marked known=false | consistent wit
```

4. Controlled Tamper Test & Refusal Gate:

```
# Tamper with log A
Add-Content evidence\camera_console_synthetic_A.log "# edit"

# Verify detects tamper
python evidence_tool.py verify
MISMATCH  camera_console_synthetic_A.log
OK        camera_console_synthetic_B.log
OVERALL: FAIL - evidence changed, missing, or unlisted   (Exit code: 1)

# Parse refuses execution
python evidence_tool.py parse --file evidence/camera_console_synthetic_A.log
File: camera_console_synthetic_A.log
SHA-256 now : e10a4ebdcc2beec214ac697d5f20c6ced73533fdddcbe2e8c78eaae7306adf21
Integrity   : MISMATCH
Refusing to parse. Re-run 'verify', or use --allow-modified (this will be logged). (Exit code: 1)

# Restoration from clean backup
Copy-Item ..\backup_A.log evidence\camera_console_synthetic_A.log -Force
python evidence_tool.py verify
OK        camera_console_synthetic_A.log
OK        camera_console_synthetic_B.log
OVERALL: PASS - all hashes match   (Exit code: 0)
```

6. Limitations

1. **Synthetic Data Constraints:** All logs, network flows, and session IDs analyzed are synthetic. Real IoT embedded systems exhibit asynchronous multi-threaded kernel noise, binary logging formats, and non-linear clock drift that require specialized reverse engineering.
2. **Hardware Hypothesis Untested (H1 Open):** Because live physical hardware was not acquired or probed, hypothesis **H1 (Device Compromise) remains untested rather than disproven**. The absence of compromise indicators in high-level logs does not prove local security.
3. **Cryptographic Scope:** A matching SHA-256 hash confirms that a file has remained unaltered *since the moment it was first hashed*; it cannot prove that the file was authentic or uncompromised when initially generated on the device.
4. **Heuristic Rule Boundaries:** The four rules in `evidence_tool.py` represent demonstration detection patterns, not a complete intrusion detection engine. Attackers can masquerade processes under allowed names (e.g., `camerad`).
5. **Plain JSON Log Security:** Storing the audit trail in a local plaintext JSON file is vulnerable to manipulation if the analyst workstation is compromised. In production forensics, logs must be written to append-only write-once media or a remote SIEM.

7. Qualified Conclusion

Based strictly on the synthetic artifacts evaluated in the Section 4 Evidence Matrix:

- **Hypotheses Cannot Be Definitive:** The available evidence cannot definitively isolate one single hypothesis among the four possibilities.
- **Cloud-Side Leaning (Low Confidence):** The technical facts confirm that the motion alert setting was altered through a command issued via the cloud API (`CMD_RECEIVED` matching `CONFIG_CHANGE` at 02:10:07) following a newly authenticated session (`S-0002` at 23:55:02). This sequence is slightly more consistent with cloud-side account interaction (**H2: Cloud Account Misuse** or **H4: Authorized User Action**) than with an isolated on-device hardware compromise (**H1**).
- **Device Compromise (H1) and Malfunction (H3) Not Excluded:** Because volatile RAM and SPI flash were not extracted, H1 is **untested, not excluded**. Similarly, backend cloud automation or synchronization failures (H3) could account for the setting change without human user interaction.
- **The 4-Minute Gap is Not Proof of Causation:** The 4-minute 4-second interval between the setting change (02:10) and the first video upload (02:14) represents a temporal correlation only. It does not establish that the setting change caused the uploads or that the same actor initiated both events.
- **Attribution Boundaries:** The findings do not name any individual, establish motive, or determine criminal intent. Forensic evidence demonstrates what state transitions occurred on the system; attribution of authorship requires corroborating external records.

8. Verified References

No.	Full Citation	Primary Application	Verification Status
1	**NIST SP 800-86:** *Guide to Integrating Forensic Techniques into Incident Response*. Kent et al., NIST, 2006. DOI: 10.6028/NIST.SP.800-86	Data acquisition from multiple source types	VERIFIED (NIST CSRC)
2	**NIST SP 800-101 Rev. 1:** *Guidelines on Mobile Device Forensics*. Ayers et al., NIST, 2014. DOI: 10.6028/NIST.SP.800-101r1	Companion mobile app extraction planning (E5)	VERIFIED (NIST CSRC)
3	**ISO/IEC 27037:2012:** *Guidelines for identification, collection, acquisition and preservation of digital evidence*. ISO/IEC, 2012.	Preserving digital evidence integrity and chain-of-custody	VERIFIED (ISO Catalogue)
4	**IETF RFC 3227:** *Guidelines for Evidence Collection and Archiving*. Brezinski & Killalea, 2002.	Order of volatility hierarchy (E1 to E6)	VERIFIED (IETF RFC Editor)
5	**IETF RFC 2606:** *Reserved Top Level DNS Names*. Eastlake & Panitz, 1999.	Fictional `.example` domain usage	VERIFIED (IETF RFC Editor)
6	**IETF RFC 5737:** *IPv4 Address Blocks Reserved for Documentation*. Arkko et al., 2010.	Documentation IP address ranges (`192.0.2.0/24` , `203.0.113.0/24`)	VERIFIED (IETF RFC Editor)
7	**NIST FIPS PUB 180-4:** *Secure Hash Standard (SHS)*. NIST, 2015. DOI: 10.6028/NIST.FIPS.180-4	Cryptographic SHA-256 specification	VERIFIED (NIST FIPS)
8	**NIST SP 800-30 Rev. 1:** *Guide for Conducting Risk Assessments*. NIST, 2012. DOI: 10.6028/NIST.SP.800-30r1	Likelihood and impact scoring methodology	VERIFIED (NIST CSRC)
9	**Python 3 Documentation:** *The Python Standard Library* (v3.11.9). Python Software Foundation.	Technical implementation of `evidence_tool.py`	VERIFIED (docs.python.org)
10	**Utility Documentation:** *flashrom* manual, *picocom* documentation, *Wireshark* manual.	Planned serial and flash extraction procedures	VERIFIED (Official docs)

9. AI-Use Disclosure & Student Declaration

9.1 AI Assistance Disclosure

Generative AI assistance (Claude / Anthropic and Gemini / Google) was utilized in drafting the scenario structure, initial boilerplate code for argument parsing, and standard reference identification. I independently

configured the Python environment, executed the full tool run and tamper test on my machine, verified all outputs and SHA-256 hashes, checked reference citations against official portals, and authored the analytical evaluations and qualified conclusions.

Student Declaration

*"I, **DEEPTANSH NAGAR** (Student ID: **500121033**), hereby declare that this submission represents my own individual academic work for Project Code **DF-PBL-026** (IoT Device Forensic Plan). All technical commands, terminal outputs, cryptographic hashes, and tamper tests documented in Section 5 were personally executed and validated on my local workstation. I independently checked all literature citations, evaluated the forensic matrix without assuming guilt or intent, and can explain, demonstrate, and defend every function in `evidence_tool.py` and every decision in this investigation memo."*

Signed: Deeptansh Nagar **Date:** October 27, 2026 **Program:** B.Tech Digital Forensics & Cyber Security